Методические указания к лабораторной работе №4

Тема: «Основы работы с системой контроля версий Git и веб-сервисом GitHub»

Цель работы: Изучить базовые концепции контроля версий, приобрести практические навыки установки и настройки Git, создания локальных и удаленных репозиториев, а также организации структуры для хранения последующих лабораторных работ по программированию.

Продолжительность: 2 академических часа.

Краткие теоретические сведения

Git — это распределенная система контроля версий. Она позволяет фиксировать изменения в файлах (делать «снимки» проекта — коммиты), возвращаться к предыдущим версиям кода и безопасно вести разработку.

- **Локальный репозиторий** — скрытая папка `.git` на вашем компьютере, где хранится вся история изменений вашего проекта.
- **Удаленный репозиторий** — копия вашего проекта, размещенная на сервере в сети (в данном случае на GitHub). Она служит для резервного копирования и демонстрации кода.
- **Коммит (Commit)** — сохраненное состояние проекта с уникальным идентификатором. Каждое сохранение обязательно должно сопровождаться осмысленным описанием выполненной работы.

Пошаговое выполнение работы

Шаг 1. Установка Git на компьютер

1. Скачайте установочный файл Git для вашей операционной системы с официального сайта: git-scm.com.
2. Запустите установщик. Для всех параметров можно оставить настройки по умолчанию (нажимайте *Next* / *Далее*).
3. Проверьте корректность установки. Откройте терминал (в Windows — **Командную строку** или **Git Bash**, в macOS/Linux — **Терминал**) и введите команду:

   ```
   git --version
   ```

   Если утилита установлена правильно, на экране отобразится текущая версия программы.

Шаг 2. Глобальная настройка Git

Перед созданием первого репозитория необходимо указать Git ваше имя и адрес электронной почты. Эти данные будут привязываться к каждому вашему коммиту. В терминале выполните две команды

(вместо параметров в кавычках подставьте свои данные):

```
git config --global user.name "Иван Иванов"
git config --global user.email "ivan.ivanov@example.com"
```

Рекомендуется использовать тот же email, на который вы зарегистрируете (или уже зарегистрировали) аккаунт на GitHub.

Шаг 3. Создание структуры локального репозитория

Вам необходимо создать единую корневую папку, внутри которой вы будете хранить все лабораторные работы по текущей дисциплине.

1. Создайте папку на компьютере (например, `programming-labs`) и перейдите в неё через терминал:

   ```
   mkdir programming-labs
   cd programming-labs
   ```

2. Инициализируйте Git-репозиторий в этой папке:

   ```
   git init
   ```

   Внутри папки появится скрытая директория `.git`. С этого момента Git отслеживает изменения в этой папке.

3. Создайте правильную структуру каталогов. Внутри корневой папки создайте директорию для первой лабораторной работы и пустой файл отчета или кода:

   ```
   mkdir lab_01
   ```

4. Внутри папки `lab_01` создайте ваш рабочий файл, например `main.py` (или `index.html` / `task.txt` в зависимости от вашего предмета).

Шаг 4. Фиксация изменений (Первый коммит)

1. Проверьте состояние вашего репозитория:

   ```
   git status
   ```

   Вы увидите, что папка `lab_01` подсвечена красным цветом как неотслеживаемая (Untracked).
2. Подготовьте все созданные файлы к сохранению:

```
git add .
```

3. Сделайте первый коммит, зафиксировав создание структуры проекта:

```
git commit -m "Initial commit: create structure and lab_01"
```

Шаг 5. Создание общедоступного репозитория на GitHub

1. Зайдите на сайт github.com и войдите в свой аккаунт (или зарегистрируйтесь, если у вас его еще нет).
2. В правом верхнем углу нажмите на иконку `+` и выберите `New repository` (Новый репозиторий).
3. Настройте репозиторий:
   - **Repository name:** Укажите имя, совпадающее с вашей локальной папкой (например, `programming-labs`).
   - **Public/Private:** Обязательно выберите **Public** (Общедоступный), иначе преподаватель не сможет увидеть ваш код по ссылке.
   - **Инициализация репозитория:** Все галочки (*Add a README file*, *Add .gitignore*, *Choose a license*) оставьте **пустыми**. Это критически важно, так как у нас уже есть локальный код.
4. Нажмите кнопку `Create repository`.

Шаг 6. Связывание локального репозитория с GitHub и отправка кода

После создания репозитория GitHub покажет страницу с подсказками. Найдите блок команд под заголовком **«...or push an existing repository from the command line»** (отправка существующего репозитория из командной строки).

Скопируйте и поочередно выполните эти три команды в своем терминале (находясь в папке `programming-labs`):

```
git remote add origin https://github.com/ВАШ_НИКНЕЙМ/programming-labs.git
git branch -M main
git push -u origin main
```

Примечание: При первом выполнении команды `git push` система может открыть окно браузера для авторизации на GitHub. Подтвердите доступ.

Обновите страницу вашего репозитория на GitHub в браузере — там должна появиться структура вашей папки `lab_01`.

Шаг 7. Markdown

Отчет к лабораторной работе №3 и последующие отчеты выполнить в формате Markdown.

Порядок сдачи лабораторной работы

Для проверки выполненной работы вам необходимо предоставить преподавателю ссылку на ваш созданный публичный репозиторий GitHub.

Формат ссылки: `https://github.com/ВАШ_НИКНЕЙМ/programming-labs`

> **Важно:** Все последующие лабораторные работы в течение семестра вы будете выполнять в этой же локальной папке (создавая каталоги `lab_02`, `lab_03` и т.д.), делать коммиты и отправлять их на GitHub одной командой `git push`. Ссылка на ваш репозиторий при этом не меняется, преподаватель всегда сможет увидеть актуальное состояние ваших работ.

Контрольные вопросы для защиты работы

1. В чем разница между локальным репозиторием Git и удаленным репозиторием GitHub?
2. За что отвечают команды `git add` и `git commit`? Какова последовательность их применения?
3. Что делает команда `git push`?
4. Зачем нужна команда `git status`?

Источники

1. Pro Git. Scott Chacon, Ben Straub. - 2026. <файл: progit.pdf> <ссылка: https://git-scm.com/book/ru/v2>